

Security Monitoring & AI Automation Home Lab

Building and evaluating an AI-assisted incident triage pipeline on a hybrid SIEM
Project Review - Roger (R.J.) Rolander - 2026

This is a review of a security operations project that I built and evaluated end-to-end. It is written to be useful to two readers at once: a non-technical reader who wants to understand what was built, and why it matters, and a technical reader who wants to see the engineering decisions and verify the results.

Executive Summary

Modern organizations generate an enormous amount of security data. Authentication attempts, process executions, account changes, administrative actions, and thousands of other events are continuously collected and analyzed by detection platforms designed to identify suspicious behavior. The problem is that these systems often generate more alerts than security analysts can realistically investigate in depth. As a result, many organizations have begun to explore the use of AI (Artificial Intelligence) to help prioritize and triage incidents. The challenge is that an AI system that produces confident but unsupported conclusions can be more dangerous than no AI at all, because analyst trust depends on the ability to verify the reasoning behind every recommendation.

I built a working security operations lab and added an AI triage layer on top of it, but the goal was never simply to connect an AI to a security tool. The goal was to determine whether the AI’s conclusions could actually be trusted, and to build an evaluation rigorous enough to detect when they could not.

It did exactly that. During testing, the AI correctly refused to make a claim that the available evidence could not support. That result led to the discovery of a defect in my own data pipeline that had been silently feeding incomplete information into every incident. I traced the issue to its root cause, fixed it, and verified the result. An evaluation that only confirms success proves very little. This one demonstrated its value by identifying a real failure.

Tools & Technologies

Cloud & SIEM	Microsoft Azure, Azure Arc, Azure Monitor Agent, Data Collection Rules, Log Analytics, Microsoft Sentinel, Microsoft Defender portal
Detection & Query	KQL (Kusto Query Language), MITRE ATT&CK framework
Infrastructure	VirtualBox, Windows Server 2025 (Active Directory Domain Services), Kali Linux
Attack Simulation	netexec, impacket, evil-winrm
AI Pipeline	Python, Sentinel REST API, Log Analytics API, Large Language Model (Claude Opus)
Development	Git, GitHub, Claude Code

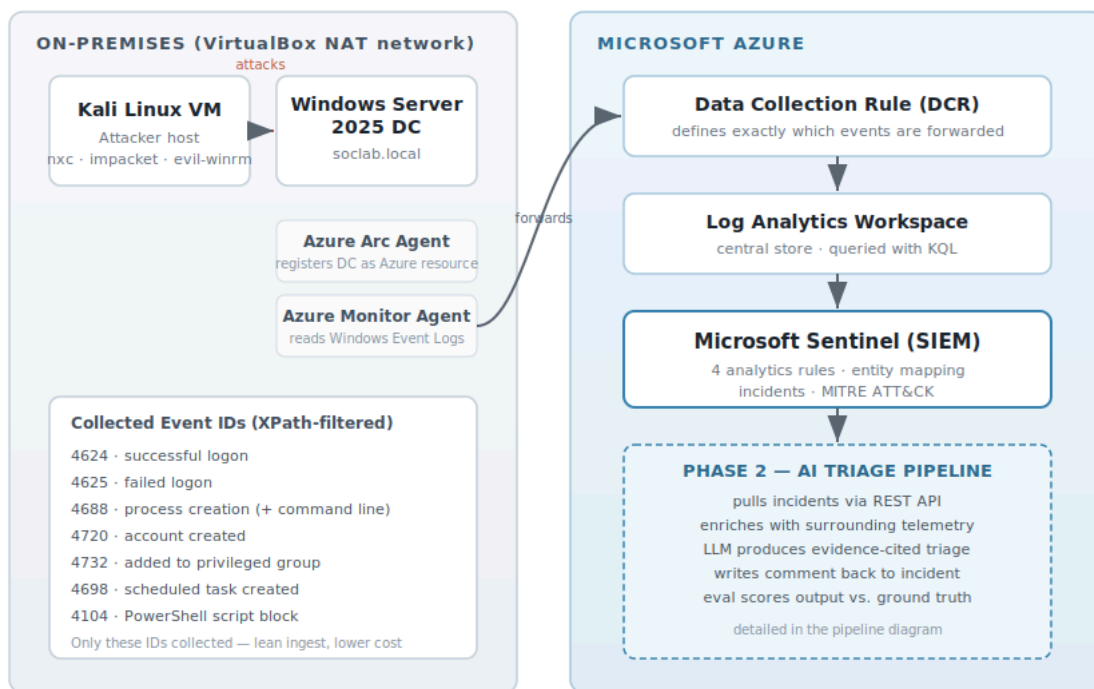
Lab Overview

The foundation of this lab is a hybrid security operations environment. In essence, it is a miniature version of the setup a real organization uses to watch for attacks: a server that generates security logs, a system that collects and stores those logs in the cloud, and detection rules that raise an alarm when something suspicious happens.

There is one deliberate design choice worth noting because it reflects how real organizations are structured. Most practice labs create their server directly inside the cloud provider. I instead ran the server locally on my computer and connected it to the cloud through Microsoft's Azure Arc, the same mechanism enterprises use to bring their existing on-premises and non-cloud systems under cloud monitoring. This was a difficult task, and it exposed a series of real onboarding and authentication problems. Those problems, and how I diagnosed each one, are documented in a troubleshooting log that a technical reviewer can read.

Lab Architecture: On-Premises to Cloud SIEM

Local virtualized domain controller onboarded to Azure via Arc, feeding Microsoft Sentinel



On top of this foundation I wrote four detection rules, each mapped to the industry-standard MITRE ATT&CK framework, which is the shared vocabulary security teams use to name attacker techniques. The rules cover the common signals of an intrusion. These include repeated failed logins (a brute-force attempt), a user being added to a powerful administrative group, suspicious PowerShell commands, and a multi-stage rule that connects two separate events, a new account being created and then rapidly granted administrator rights, all into a single correlated alert.

A detail that will matter later:

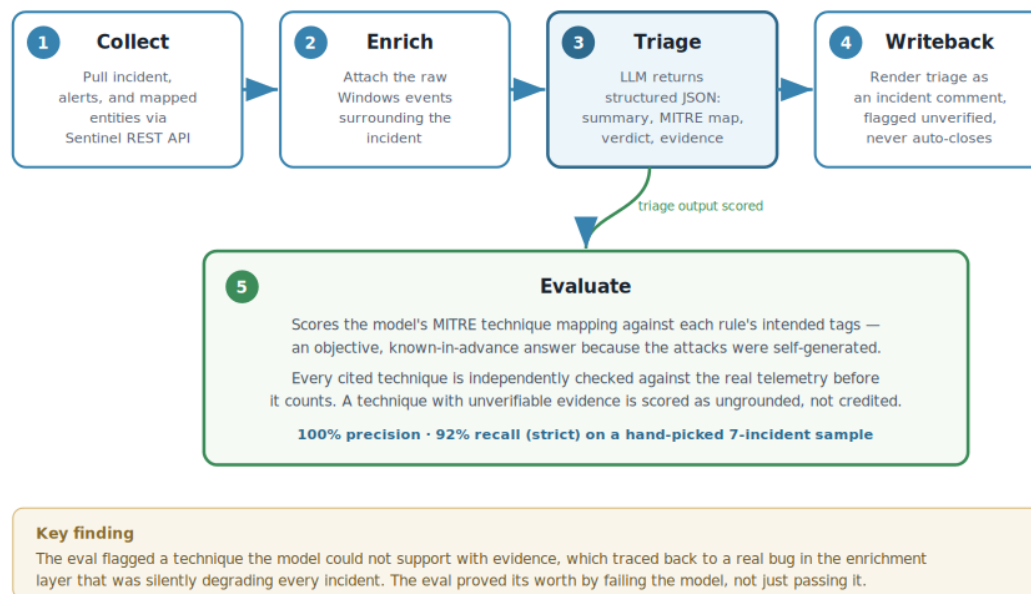
The correlation rule links the “account created” and “account elevated” events using each account’s *security identifier* (its SID), a permanent internal ID, rather than its username. This is the correct engineering choice because the username is not reliably present on the elevation event. This distinction, ID versus name, became the root of a bug found later, which is a clean illustration of the same principle mattering in two places.

Phase 2: The AI Triage Pipeline

With a working detection platform generating real incidents, the second phase added the AI layer. It is a five-stage pipeline. Each stage does one job, and each was built and tested on its own before the next was started, so that a failure anywhere could be isolated quickly.

The AI Triage Pipeline

Five stages, each independently runnable and testable before the next



The first four stages do the intuitive work. The pipeline **collects** an incident and its details from the security platform, **enriches** it by attaching the surrounding raw security events so the AI has real evidence to reason over rather than just a headline, sends that package to a large language model for **triage**, and then **writes** the result back as a comment on the incident. Two guardrails are built in deliberately. Every AI-written comment is clearly labeled as automated and unverified, and the system never closes an incident on its own.

A human is always the one making an actual decision while the AI only ever recommends.

Key Findings

The most important result of this project was not a score. It was what happened when the evaluation was run against the correlation incident, the one where a new account is created and then rapidly elevated to administrator.

The AI examined the evidence it was given and reported the elevation technique at *low confidence*, explicitly noting that the event proving the elevation was not present in the data it received. It refused to assert something it could not support. That refusal was correct behavior, and it was also a signal that something upstream was wrong, because that elevation event definitely happened.

Tracing it back revealed two separate problems in the enrichment stage, the step that gathers the surrounding evidence:

- **The wrong lookup key.** The elevation event refers to the account by its internal security ID, not its username, but the enrichment was searched only by username. It could never have found the elevation for the very account being elevated. This is the same ID-versus-name distinction the correlation detection rule itself already handled correctly, now appearing as the cause of a bug one layer down.
- **Evidence starved by noise.** The enrichment collected only the first hundred events in a window, in time order, and on a busy host those hundred slots were used up by routine background activity before the query ever reached the moment the incident actually occurred. Across the collected incidents, this was silently affecting the large majority of them.

After discovering this bug, I re-gathered the evidence for every incident against the still accessible logs, and re-ran the triage. The same incident that had produced a low-confidence result now produced a confident, fully evidenced one, citing the exact elevation event and the specific administrative command that caused it. The before-and-after is concrete:

	Before the fix	After the fix
AI Confidence	Low	High
Cited Evidence	None found in the data. Only the alert's own description	The exact elevation event, plus the command that ran it
Evaluation Verdict	Ungrounded. Scored as a miss	Grounded. Scored as a true positive

What This Demonstrates

The transferable skills here are less about the specific tools and more about the approach:

- **Building a real-world standard, not a demo standard.** The hybrid cloud onboarding, the production-shaped detection rules with tuned false-positive handling, and the human-in-the-loop guardrails all reflect how this work is done in a real environment rather than a tutorial.
- **Integrating complex systems.** The project required connecting on-premises infrastructure, cloud services, detection logic, APIs, and an AI model into a single workflow while ensuring that data moved reliably between each stage.
- **Measuring instead of asserting.** The whole project is organized around the question “how would I know if this actually works,” and the answer is a repeatable evaluation with an honest, defensible number.
- **Diagnosing the root cause.** The central finding is a debugging story. I got an unexpected result, traced through layers, to two distinct root causes, fixed them, and verified it.
- **Communicating to mixed audiences.** The same project is documented as a plain-language overview, a technical detection write-up, a troubleshooting log with verbatim errors, and a rigorous evaluation report, each pitched at the read who needs it.
- **Validating AI outputs instead of blindly trusting them.** Rather than treating AI as an authority, the project evaluated whether every conclusion could be supported by evidence from the underlying telemetry.
- **Communicating with mixed audiences.** The same project was documented as a plain-language overview, a technical detection write-up, a troubleshooting log with verbatim errors, and a rigorous evaluation report, with each document tailored to a different audience.